NUMARA / P11: Context-Gated Convergent Attractor Routing for Candidate-Evidence Workflows

Draft scientific paper v0.1 - local harness results through V0.3.2

Salvador Sena / @schicanoloco, with AI-assisted drafting and harness generation

Interpretation boundary: local harness results for a research prototype; not a production benchmark.

Abstract

This draft introduces NUMARA / P11, a prototype architecture for context-gated convergent routing in AI-assisted workflows. The central hypothesis is that useful outputs can be treated as candidate attractors when different heuristic paths converge on the same invariant structure, but only when the task class makes that invariant eligible. The architecture combines a primitive basis, symbolic topology, heuristic path fields, task gates, an attractor-style activation engine, negative controls, memory/output handling, and audit/rollback logs.

A sequence of local harness experiments showed a useful experimental arc: V0.1 passed a deterministic sanity check; V0.2 preserved transfer under noise but exposed over-convergence and false convergence; V0.3 repaired the observed false-convergence failure using explicit task gates and context-negative suppression; and V0.3.2 validated the V0.3 structure and refactored it into a package-style implementation scaffold. These results are promising for research and assisted-authoring workflows, but remain bounded local harness results requiring multi-seed, cross-task, external validation before stronger claims can be made.

Keywords: AI routing; candidate evidence; context gating; convergent attractors; heuristic path fields; negative controls; RAG; ReAct; MCP; LLM routers; audit and rollback.

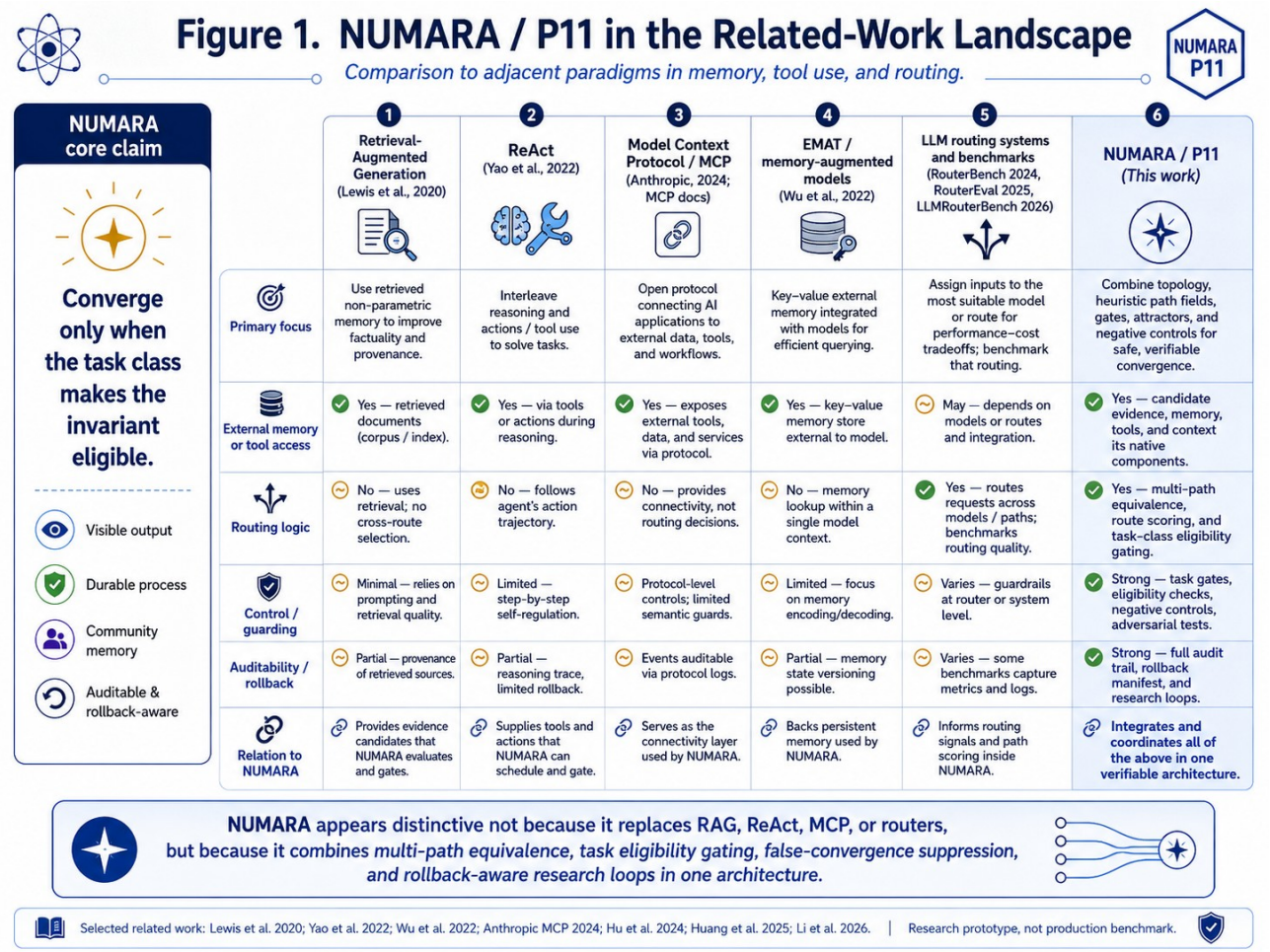

Figure 1. Related-work landscape. NUMARA / P11 is positioned as a coordinating architecture that can integrate memory, tools, routing, gates, and audit loops rather than replacing adjacent paradigms.

1. Introduction

Modern AI systems increasingly combine language models with retrieval, tools, external memory, connectors, evaluators, and routing layers. Retrieval-augmented generation (RAG) connects generation to retrieved non-parametric memory to improve specificity, factuality, and provenance [1]. ReAct interleaves reasoning and acting so that model trajectories can call tools or external resources while solving tasks [2]. The Model Context Protocol (MCP) standardizes how AI applications connect to external systems, data sources, tools, and workflows [4,5]. Multi-LLM routers and router benchmarks study how requests can be assigned to suitable models or routes for performance-cost tradeoffs [6-9].

NUMARA / P11 is not proposed as a replacement for these directions. The working claim is narrower: a research prototype can coordinate several of these ideas around a context-gated convergence mechanism. The system asks whether different paths can independently converge on a similar invariant structure, whether that convergence can transfer across task families, and whether false convergence can be suppressed when the task context is wrong.

The core design rule is: P11 convergence requires eligibility. Equivalence without context gating becomes false convergence.

2. Related Work and Positioning

RAG provides a foundation for grounding generated text in retrieved external context. Its original formulation combined a parametric seq2seq model with a dense vector index over a non-parametric corpus [1]. Later multilingual RAG work suggests the design space remains active and context-dependent, especially when retrieval spans languages and inconsistent sources [10].

ReAct is relevant because it treats reasoning and action as interleaved rather than separate processes. This aligns with the NUMARA idea that an output path may include reasoning-like state, tool access, guards, and update steps rather than a single prompt-response pass [2].

MCP is relevant as infrastructure rather than routing logic. It standardizes connections between AI applications and external systems, which could supply the tools, data, and workflows used by a NUMARA-style route [4,5].

Memory-augmented models such as EMAT explore efficient external memory access and integration. NUMARA is not a memory architecture in that narrow sense; instead, it treats memory as one component inside a wider control and convergence loop [3].

LLM routing benchmarks show that routing is now an active area. RouterBench introduced a benchmark for multi-LLM routing systems [6]. RouterEval expanded the problem to a large benchmark for exploring model-level routing scale [7]. LLMRouterBench provided a unified benchmark with more than 400K instances across 21 datasets and 33 models and found that many routing methods show similar performance under unified evaluation, with persistent model-recall failures still separating routers from oracle routing [8]. TwinRouterBench extends the routing problem toward step-level agentic evaluation in long-horizon applications, including static and dynamic tracks [9].

Against that landscape, NUMARA / P11 appears distinctive in combination, not in isolation. Its novelty claim should remain provisional: it combines multi-path equivalence, task eligibility gates, false-convergence suppression, negative controls, candidate-evidence boundaries, and rollback-aware research loops in one explicit architecture.

3. Architecture

The current pre-V0.4 architecture uses seven operational layers plus a primitive basis. L0 defines a primitive basis P0-P12, where P11 is equivalence. L1 defines symbolic topology with nodes and typed edges. L2 defines heuristic path fields: reusable activation corridors that bias groups of nodes without forcing a fixed route. L3 introduces task gates and eligibility checks. L4 performs weighted activation under noise. L5 applies guards and negative controls. L6 produces candidate memory/output artifacts. L7 stores audit and rollback artifacts.

The design uses a deliberately strict boundary between candidate evidence and verified truth. Convergence inside the harness can support candidate status, but it does not verify external facts. External factual claims require separate evidence and review.

The central invariant family tested in the current harness is: visible output, durable process, and community memory. In the MILPA and build-day experiments, this appeared as: the grant or build

day is the visible output; the repeatable process is the durable asset; and community memory carries continuity across events, roles, and funding cycles.

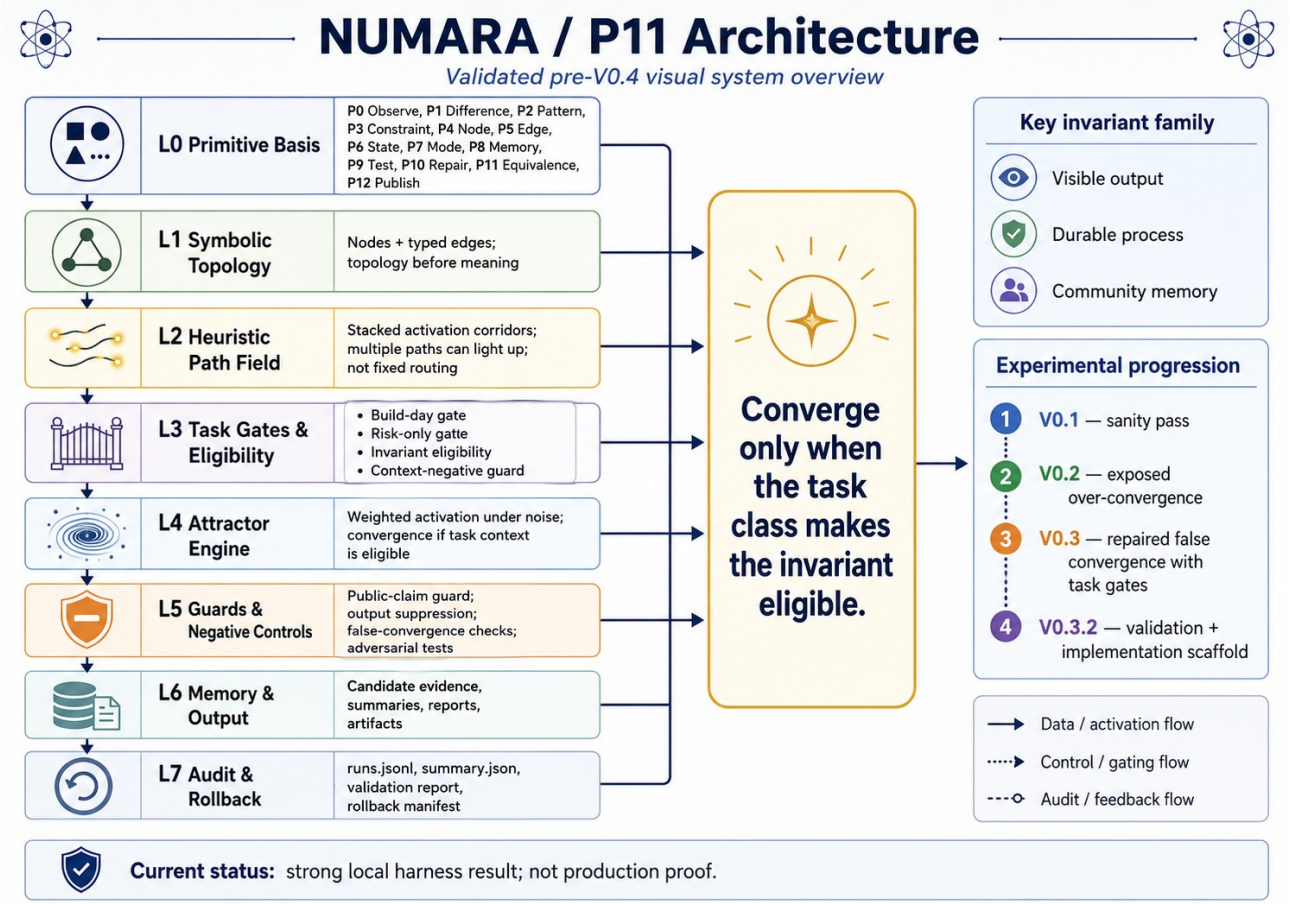

Figure 2. Architecture overview. The central design rule is that convergence is allowed only when the task class makes the invariant eligible.

4. Experimental Methods

The experiments are local stochastic harness tests, not production benchmarks. The harness uses JSON graph files, heuristic path definitions, guard rules, configuration files, randomized activation noise, edge-weight noise, edge dropout, adversarial guard tests, negative-control tests, and JSONL logging.

The experimental arc consisted of four main stages. V0.1 tested a deterministic sanity-check harness. V0.2 introduced randomized perturbation and exposed a serious over-convergence failure: the system was too eager to converge on the known invariant even when the context did not justify it. V0.3 added task-gate nodes, context-negative suppression, invariant eligibility, risk-only negative controls, and stronger final-output suppression. V0.3.2 validated V0.3 structurally and refactored the harness into a package-style implementation scaffold.

The pass criteria for V0.3 included transfer success above threshold, unsafe failure below threshold, adversarial guard pass above threshold, and false-convergence rates below threshold. The critical test was whether V0.3 could reduce false convergence without destroying useful transfer.

5. Results

Stage	Purpose	Key result	Interpretation
V0.1	Deterministic sanity check	Invariant reproduced across paths	Harness coherence; not generalization proof
V0.2	Noise and robustness pass	Transfer 81.9%; false convergence 66.7%; adversarial guard pass 0.0%	Useful failure: graph bias was too strong
V0.3	Task-gated repair	11,400 runs; transfer 99.2%; false convergence 0.0%; adversarial guard pass 100%	Known over-convergence failure repaired in local harness
V0.3.2	Validation and implementation scaffold	480-run smoke test; transfer 99.67%; false convergence 0.0%; adversarial guard pass 100%	Refactored implementation can rerun mechanism

The most important result is not simply that V0.3 achieved higher scores. The important result is the experimental loop: V0.2 failed in a meaningful way, the failure was diagnosed as over-convergence, V0.3 introduced a targeted mechanism to address it, and the invariant survived the repair.

This supports a provisional design rule: useful convergence appears strongest when paired with task gating, negative controls, and explicit guard suppression.

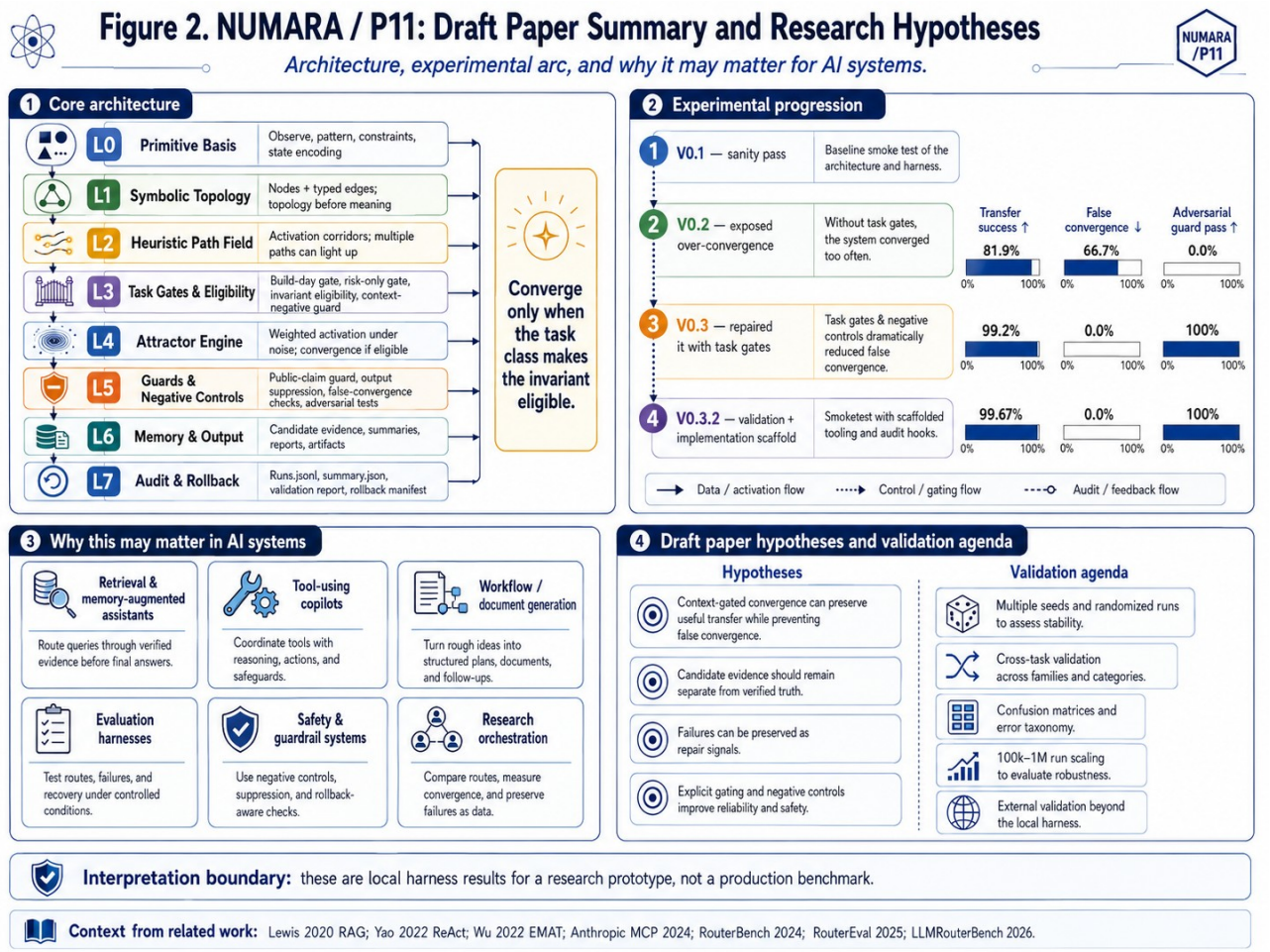

Figure 3. Draft paper summary and hypotheses. The figure ties the architecture, experimental progression, application domains, and validation agenda into one paper-facing visual.

6. Comparison to Emerging Router and Agent Systems

LLM routers generally ask which model, model tier, or route should handle an input. RouterBench, RouterEval, LLMRouterBench, and TwinRouterBench formalize different aspects of that problem, including cost-performance tradeoffs, large-scale model candidate pools, unified evaluation, and agentic step-level routing [6-9]. NUMARA / P11 can use routing ideas, but it is not limited to choosing a model. It also asks whether a route should be eligible, whether multiple paths converge on the same structure, whether that convergence is a false positive, whether output should be suppressed, and whether the run should become candidate evidence or a failure record.

RAG and EMAT supply relevant external memory and knowledge mechanisms [1,3]. ReAct and MCP supply relevant tool-use and connectivity paradigms [2,4,5]. NUMARA / P11 is best framed as a coordinating architecture that can sit above or alongside such components. It could use RAG as evidence supply, MCP as the connector layer, ReAct-like trajectories as action paths, memory-augmented models as persistent memory, and router benchmarks as evaluation inspiration.

The plausible distinction is architectural integration: NUMARA combines topology, heuristic path fields, task-class eligibility, attractor-style convergence, guard suppression, negative controls,

audit/rollback, and candidate-evidence management. That integrated control loop is the current research object.

7. Potential AI Applications

If validated beyond the local harness, NUMARA-like control could help in several areas: retrieval and memory-augmented assistants, tool-using copilots, workflow and document generation, agent evaluation harnesses, safety and guardrail systems, and research orchestration. The common theme is controlled transfer: preserve useful repeated structure while preventing context-mismatched convergence or unsafe publication.

The strongest near-term use case is assisted authoring and research workflow support. The system is already useful for organizing drafts, preserving failure records, generating structured follow-up packets, and preventing candidate ideas from being mistaken for verified truth. Higher-stakes domains would require stricter external validation and domain-specific evidence rules.

Figure 4. Potential AI applications and research outlook. The application space overlaps with RAG, tool-use, routing, evaluation, guardrails, and research orchestration.

8. Limitations

The current results are local harness results. They are not production benchmarks. The graph, task families, invariant family, and negative controls are still hand-designed. The high V0.3 scores

therefore show that the repair works in the current constructed setting, not that it will generalize broadly. The system also risks overfitting to the invariant family unless V0.4 expands task families and negative controls.

Another limitation is that convergence can feel like truth. The architecture must preserve a hard boundary: internal convergence evidence can only support candidate status, not verification. Verification must come from independent evidence, domain review, and external tests.

A further limitation is operational cost. Many layers, guard rules, and logs create maintenance overhead. The practical benefit depends on whether the architecture improves output quality, traceability, and failure recovery enough to justify that complexity.

9. V0.4 Experimental Plan

The next experimental step is V0.4: multi-seed cross-task validation. It should include multiple seeds, multiple task families, per-task eligibility maps, risk-only negative controls, confusion matrices, false-positive and false-negative rates, CSV summaries, automatic report generation, and scaling toward 100K to 1M runs. The central question is whether the V0.3 repair survives a broader distribution of tasks and negative controls.

V0.4 target	Rationale
Multiple random seeds	Detect seed sensitivity and stability problems
Multiple task families	Test whether gating generalizes beyond build-day / MILPA framing
Eligibility maps	Define which invariants are allowed for which task classes
Confusion matrix	Measure false positives, false negatives, true positives, and true negatives
100K-1M run scaling	Stress the harness and identify rare failures
External review	Separate internal convergence from real-world validity

10. Conclusion

NUMARA / P11 is a promising research prototype for context-gated convergence. The strongest current evidence is the failure-repair loop from V0.2 to V0.3: the system exposed false convergence, introduced task gates and context-negative controls, and preserved useful transfer while suppressing the observed failure. This does not prove broad generalization. It does, however, justify continued testing and implementation hardening.

The working scientific claim should remain measured: NUMARA / P11 may help coordinate memory, routing, tool use, guardrails, and audit loops by treating convergence as candidate evidence only when task eligibility conditions are met. The next step is V0.4 multi-seed, cross-task validation.

References

[1] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." arXiv:2005.11401, 2020. https://arxiv.org/abs/2005.11401

[2] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629

[3] Yuxiang Wu et al. "An Efficient Memory-Augmented Transformer for Knowledge-Intensive NLP Tasks." arXiv:2210.16773, 2022. https://arxiv.org/abs/2210.16773

[4] Anthropic. "Introducing the Model Context Protocol." November 25, 2024. https://www.anthropic.com/news/model-context-protocol

[5] Model Context Protocol documentation. "What is the Model Context Protocol (MCP)?" https://modelcontextprotocol.io/docs/getting-started/intro

[6] Qitian Jason Hu et al. "RouterBench: A Benchmark for Multi-LLM Routing System." arXiv:2403.12031, 2024. https://arxiv.org/abs/2403.12031

[7] Zhongzhan Huang et al. "RouterEval: A Comprehensive Benchmark for Routing LLMs to Explore Model-level Scaling Up in LLMs." arXiv:2503.10657, 2025. https://arxiv.org/abs/2503.10657

[8] Hao Li et al. "LLMRouterBench: A Massive Benchmark and Unified Framework for LLM Routing." arXiv:2601.07206, 2026. https://arxiv.org/abs/2601.07206

[9] Pei Yang et al. "TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing." arXiv:2605.18859, 2026. https://arxiv.org/abs/2605.18859

[10] Leonardo Ranaldi, Barry Haddow, and Alexandra Birch. "Multilingual Retrieval-Augmented Generation for Knowledge-Intensive Task." arXiv:2504.03616, 2025. https://arxiv.org/abs/2504.03616

Appendix A: Harness Artifacts

Primary artifact packages generated in this work include the V0.1, V0.2, V0.3, and V0.3.2 harness packages. Key files include graph.json, paths.json, guards.json, config.json, tests.yaml, runner.py, results.json, runs.jsonl, batch_report.md, validation_report.md, implementation_report.md, and rollback_manifest.json.